DART-VLN: Test-Time Memory Decay and Anti-Loop Regularization for Discrete Vision-Language Navigation

Shaoheng Zhang[1], Zhichen Li[1], Guangfu Ma[1,*], and Jie Mei[1,*]
[1]Harbin Institute of Technology, Shenzhen
Guangdong, China

arXiv:2607.01043v4 [cs.RO] 13 Sep 2026

***Abstract*— Memory-based agents for discrete vision-language navigation (VLN) operate under partial observability and may exhibit systematic inference-time failures even with strong pretrained backbones. This paper addresses two recurring problems: stale historical evidence during memory readout and inefficient local backtracking during action selection. We present DART-VLN, a training-free inference-time framework that combines Test-Time Memory Decay, which reweights stale and redundant memory slots without modifying their stored content, with Anti-Loop Regularization, a lightweight next-hop penalty that discourages immediate reversals. DART-VLN introduces no learnable parameters and leaves the navigation backbone unchanged. Experiments on R2R and REVERIE showed that memory decay preserved or improved task performance while reducing runtime. The addition of anti-loop regularization further shortened trajectories, reduced local backtracking, and achieved the best overall balance between navigation quality and efficiency among the evaluated GridMM variants. These results indicate that lightweight inference-time control can improve the reliability and efficiency of memory-based discrete VLN without retraining.**

***Index Terms*— Vision-Language Navigation, Discrete Navigation, Test-Time Control, Memory Decay, Anti-Loop Regularization**

I. Introduction

Language-guided embodied navigation requires reliable sequential decision-making under partial observability. Vision-language navigation (VLN) requires an embodied agent to navigate an environment by grounding language instructions in visual observations [1], [2]. Among existing formulations, discrete VLN is especially attractive because it operates over explicit viewpoint graphs and supports controllable stepwise decision-making [3]–[5].

Recent advances in VLN have been driven by pretrained navigation models, explicit memory and map representations, larger training resources, and improved recovery and planning mechanisms [3], [6]–[10]. Together, these advances have substantially improved long-horizon reasoning and benchmark performance. Yet a practical gap remains at inference time: even strong memory-based discrete VLN backbones can behave unreliably when deployed with frozen parameters. Further gains often require retraining, architectural redesign, or heavier planning modules, making them less suitable when the goal is to improve a pretrained navigator without modifying its architecture or training procedure.

In this setting, two recurring failure modes become particularly important. The first appears at memory readout. Explicit navigation memory helps agents reason over longer trajectories by storing previously visited viewpoints, visual features, or map-level context [3]–[5], [8], [11]–[13]. However, as navigation proceeds, stale or repeatedly observed evidence can persist in memory after its utility has diminished, introducing noise into memory aggregation at decision time. The second appears at action selection. Even with a strong frozen backbone, agents still exhibit inefficient local behaviors such as immediate reversals and short loops [14]–[16]. These behaviors do not always prevent success, but they lengthen trajectories, waste steps, and increase runtime.

This motivates a simple question: how much can be gained from lightweight test-time control alone, without retraining the model or redesigning the navigation stack? We answer this question with DART-VLN, a training-free test-time control framework for memory-based discrete VLN. As summarized in Fig. 1, DART-VLN introduces two complementary mechanisms that operate directly inside the existing inference loop. *Test-Time Memory Decay* is a read-side reweighting rule that suppresses stale and redundant evidence during memory aggregation without rewriting stored content. *Anti-Loop Regularization* is a lightweight next-hop penalty that discourages immediate backtracking during action selection. Together, these components improve inference behavior while leaving the learned backbone untouched and introducing no new learnable parameters.

DART-VLN is designed as a plug-in test-time control layer for discrete VLN pipelines with explicit memory. In our experiments, we instantiate it on a GridMM-based navigator and evaluate it on R2R and REVERIE [3]. Across both benchmarks, read-side decay preserves or improves task performance, while anti-loop regularization reduces local reversals, trajectory length, and runtime. These results suggest that lightweight test-time control can strengthen memory-based discrete VLN without retraining.

A. Contributions

- We present DART-VLN as a training-free plug-in layer for discrete VLN pipelines with explicit memory, using read-side decay to suppress stale and redundant evidence without rewriting stored content.

This work was supported by the Shenzhen Fundamental Research Program under Grant GXWD20231129140908002.

*Corresponding authors: Guangfu Ma (magf@hit.edu.cn) and Jie Mei (jmei@hit.edu.cn).

Code is available at https://github.com/Japluto/DART-VLN.

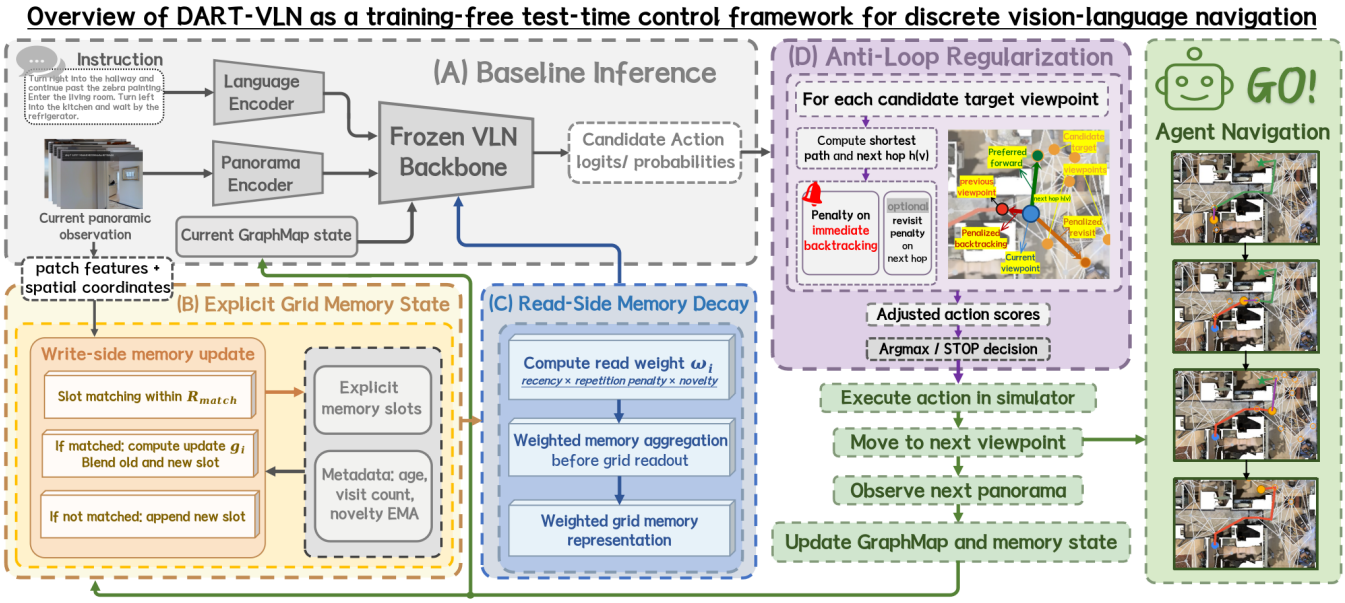


Fig. 1. Overview of DART-VLN as a plug-in test-time control layer for discrete VLN pipelines with explicit memory. Memory decay reweights historical slots at readout, while anti-loop regularization applies next-hop penalties before argmax. The backbone remains frozen with no new learnable parameters.

- We introduce a lightweight next-hop regularizer that discourages immediate backtracking and improves trajectory efficiency at inference time.
- Experiments on R2R and REVERIE with a GridMM-based navigator show that read-side decay preserves or improves task performance, while combining it with anti-loop regularization offers the best navigation-efficiency balance among the evaluated variants.

II. Related Work

Memory and map-based discrete VLN: Explicit memory and map representations support long-horizon reasoning in discrete VLN by storing visited viewpoints, aggregating graph context, or building richer topological states [3]–[5], [11], [12], [17]. As a plug-in layer, DART-VLN complements this line by controlling how existing memory is read at inference time, without introducing a new memory structure or map encoder.

Backtracking, rewind, and local recovery: A second line of work studies local navigation failures such as wrong turns, backtracking, and short loops [14]–[16]. Regret-aware, foresighted, and planning-oriented methods show that recovery behavior matters for trajectory efficiency, but often rely on learned correction policies or heavier planning modules. Our method shares this motivation but implements it as a lightweight next-hop regularizer applied directly to local action scores.

Inference-time control versus retraining-heavy improvement: Strong VLN gains also come from architectural extensions, richer scene modeling, and heavier planning [9], [10]. Although effective, these approaches can be less suitable when the goal is to improve a deployed navigator without retraining. Recent studies have also explored training-free improvements to VLN at inference time [18], [19]; DART-VLN follows this direction through read-side memory control and action-side local regularization.

III. Method

A. Problem Setting and Baseline Navigation Inference

We consider a memory-based agent operating on a discrete navigation graph $G = (V, E)$, where each node denotes a viewpoint and each edge represents a feasible transition. Given a natural-language instruction x, the agent starts from an initial viewpoint v_0 and makes sequential navigation decisions over the graph, producing a trajectory $\tau = (v_0, v_1, \ldots, v_T)$. At each step t, the agent receives the current visual observation at v_t, retrieves historical information from an explicit navigation memory, and scores the candidate actions reachable from the current viewpoint.

For evaluation, we instantiate DART-VLN on a GridMM-based discrete VLN pipeline [3]. The navigator encodes the instruction and current observation, aggregates historical evidence from explicit memory slots $\{m_i\}$, and computes action scores $s_t(v)$ for candidate target viewpoints v. This explicit-memory design provides useful long-horizon context, but stale or repeatedly observed evidence may continue to influence the current decision.

More generally, DART-VLN requires only explicit memory readout and pre-selection candidate scores, enabling plug-in use without modifying the backbone. It reweights memory slots during aggregation and regularizes candidate scores before argmax decoding. The backbone remains frozen, and no learnable parameters are introduced.

Motivated by reliable embodied navigation under partial observability [20], DART-VLN is designed for memory-based discrete VLN pipelines but validated here only on a GridMM-based navigator. The following subsections describe its read-side memory control and action-side regularization.

B. Test-Time Memory Decay

1) Motivation: In memory-based navigation, stale or repeatedly observed slots may remain active after their usefulness has faded [11], [12]. We therefore use a conservative read-side strategy: stored memory content is not rewritten, and only slot contributions are reweighted during aggregation.

2) Slot Metadata: For each memory slot m_i, we maintain three lightweight metadata variables: slot age a_i, visit count c_i, and novelty n_i. The slot age is defined as

$$a_i = t - t_i^{\text{last}},$$

where t_i^{last} is the most recent step at which slot i was refreshed. The visit count c_i records how often the corresponding region has been observed. The novelty term n_i captures recent feature change and is smoothed over time.

Concretely, let f_i^{old} and f_i^{new} denote the previous and current slot features. We first compute an instantaneous novelty score

$$\nu_i = \text{clip}\big(1 - \cos(f_i^{\text{old}}, f_i^{\text{new}}),\, 0,\, 1\big), \tag{1}$$

and then update the novelty estimate with an exponential moving average:

$$n_i^{(t)} = \rho n_i^{(t-1)} + (1-\rho)\nu_i. \tag{2}$$

Here, $\rho \in [0, 1)$ controls how strongly past novelty estimates are retained over time. In our implementation, $\rho = 0.5$. Together, these metadata provide a lightweight summary of whether a slot is recent, repeatedly observed, or still undergoing meaningful feature changes.

3) Memory Reweighting: Based on the metadata above, we assign a heuristic readout weight w_i to each memory slot. Since ν_i is clipped to $[0, 1]$ and n_i is obtained by exponential smoothing, we also have $n_i \in [0, 1]$. The readout weight is defined as

$$w_i = \text{clip}\Big(\exp(-\lambda a_i)\Big(1 - \alpha\frac{c_i}{c_i+1}\Big)(0.5 + 0.5n_i),\; w_{\min},\, w_{\max}\Big). \tag{3}$$

This formulation combines three factors. The recency term $\exp(-\lambda a_i)$ downweights slots that have not been refreshed for many steps. The repetition term $\big(1-\alpha\frac{c_i}{c_i+1}\big)$ reduces the influence of repeatedly observed regions, with α controlling the suppression strength. The novelty term $(0.5+0.5n_i)$ assigns greater weight to slots showing recent feature changes. Finally, clipping to $[w_{\min}, w_{\max}]$ prevents overly weak or aggressive reweighting.

In the default DART-VLN configuration, these weights are applied only during memory readout. The weighted memory is then passed into the original navigation pipeline without modifying the stored slot content. This keeps the intervention training-free and straightforward to isolate in ablation studies.

Algorithm 1: Default DART-VLN Inference

Notation: $h(v)$: graph next hop of candidate viewpoint v
$p_t(v)$: anti-loop penalty
$s'_t(v)$: adjusted action score
Input : Instruction x; initial viewpoint v_0; max step $T_{\max}$; memory slots $\{m_i\}$ with metadata $\{a_i, c_i, n_i\}$
Output : Navigation trajectory τ

Function `DART-VLN-Infer`$(x, v_0, \{m_i, a_i, c_i, n_i\}, T_{\max})$**:**
  Initialize $\tau \leftarrow (v_0)$ and the visit counts from the start viewpoint
  for $t \leftarrow 0$ **to** $T_{\max} - 1$ **do**
    Encode the instruction and current observation at v_t
    Compute read-side memory weights w_i using Eq. (3)
    Aggregate weighted memory and obtain candidate scores $\{s_t(v)\}$
    for each *candidate target viewpoint* v **do**
      Compute graph next hop $h(v)$ from the current viewpoint
      Compute penalty $p_t(v)$ using Eq. (4)
      Update adjusted score $s'_t(v) \leftarrow s_t(v) - p_t(v)$
    Select the next action by $\arg\max_v s'_t(v)$
    if *STOP is selected* **then**
      return τ
    Execute the transition and append the new viewpoint to τ
    Update visit counts, memory, and slot metadata for the next step
  return τ

4) Boundary of Decay-Only: In the *decay-only* setting, only the read-side reweighting in Eq. (3) is applied; stored slot representations and baseline memory updates remain unchanged. Thus, *decay-only* is a read-side denoising rule rather than a learned memory update mechanism.

5) Write-Side Memory Update Variants: Beyond read-side decay, we test a more aggressive write-side update in which matched slots are refreshed by a heuristic gate based on novelty, age, and repetition, while unmatched observations are appended as new slots. We report this as *update-only* and its combination with decay as *full-mode*. Because these variants are less stable, we include them as stress tests of more aggressive memory intervention rather than as part of the default configuration.

C. Anti-Loop Regularization for Navigation Decision-Making

1) Motivation: Immediate backtracking and short loops waste steps and expose the agent to additional error-prone decisions [14]–[16]. We therefore introduce a lightweight action-side regularizer that discourages local reversals while keeping the frozen policy otherwise unchanged.

2) Next-Hop-Based Penalty: Let $s_t(v)$ denote the original action score assigned at step t to a candidate target viewpoint v. Rather than penalizing the target directly, we examine its graph next hop $h(v)$, the first local transition on the path from the current viewpoint to v. We then define the penalty

$$p_t(v) = \beta_{\text{back}} \cdot \mathbb{I}[h(v) = v_{t-1}] + \beta_{\text{rev}} \cdot \mathbb{I}[\text{visit}(h(v)) \geq k], \tag{4}$$

and use it to adjust the action score:

$$s'_t(v) = s_t(v) - p_t(v). \tag{5}$$

The first term penalizes a next hop that returns directly to the previous viewpoint v_{t-1}. The second applies a smaller penalty to repeated revisits. Here, $\text{visit}(h(v))$ denotes prior

TABLE I

RESULTS ON THE R2R DATASET

Methods	Val Unseen					Test Unseen				
	TL↓	NE↓	SR↑	SPL↑	Runtime(s)↓	TL↓	NE↓	SR↑	SPL↑	Runtime(s)↓
VLNBERT [6]	12.03	3.93	63	57	–	12.37	4.11	63	57	–
AirBERT [7]	11.78	3.99	62	56	–	12.40	4.15	62	57	–
SEvol [10]	12.22	3.96	62	57	–	13.42	4.17	62	57	–
HOP [21]	12.28	3.81	64	57	–	12.71	3.87	64	59	–
HAMT [8]	**11.44**	2.31	66	61	–	12.27	3.95	65	60	–
TD-STP [22]	–	3.23	70	63	–	–	3.72	67	61	–
DUET [4]	13.92	3.33	72	60	–	14.71	3.63	69	59	–
BEVBert [5]	14.52	2.83	74	64	–	15.88	3.15	73	62	–
GridMM (baseline) [3]	13.27	2.83	74	64	937.99	14.43	3.35	73	62	2312.52
update-only	13.92	2.86	75	63	891.12	16.2	3.37	72	59	1725.22
decay-only	13.29	**2.59**	**76**	65	742.61	14.52	**3.19**	73	62	1621.34
full-mode	13.77	2.80	75	64	1000.84	15.95	3.30	72	60	2475.63
decay+anti-loop	**12.41**↓	2.69	**76**↑	**66**↑	**666.37**↓	**13.80**↓	3.38	**74**↑	**63**↑	**1329.56**↓

Note: Runtime is reported only for GridMM and its variants under the same implementation and hardware; cross-paper runtime is not directly comparable.

visits to the next-hop viewpoint, β_{rev} controls the revisit penalty, and k is its threshold. In the default setting, this term remains small and is applied only after repeated revisits, with $k = 2$. Immediate-backtracking suppression therefore remains the module's dominant effect.

This next-hop formulation is important because it targets the local transition actually executed at the current step, rather than only adjusting a more distant target score. It therefore aligns the regularizer with the trajectory-level behavior we aim to change.

3) Inference Scope: The anti-loop module is applied only at test time and targets deterministic, argmax-based navigation. In the default configuration, the anti-loop module operates alongside read-side memory decay and adjusts candidate scores before action selection. The module modifies neither the stop head nor the learned backbone and requires no additional planner. Its scope is deliberately narrow: it is a graph-local regularizer that improves trajectory efficiency with a frozen backbone. Because the penalty is finite and no action is masked, the agent can still backtrack when the original score provides sufficiently strong evidence.

D. Design Principle: Conservative Test-Time Control

DART-VLN follows a conservative test-time control principle: read-side reweighting suppresses stale evidence without perturbing stored representations, and the action-side regularizer discourages wasteful local loops without acting as a planner. We therefore treat the more aggressive write-side variants as stress tests rather than components of the default configuration. Algorithm 1 summarizes the resulting inference flow.

IV. EXPERIMENTS

A. Experimental Setup

We evaluate DART-VLN on Room-to-Room (R2R) [1] and REVERIE [23]. For R2R, Table I reports TL, NE, SR, SPL, and runtime on *val unseen* and *test unseen*. For REVERIE, Table II reports TL, OSR, SR, SPL, RGS, RGSPL, and runtime on *val unseen*, as the official test server is unavailable. We treat SR, SPL, and RGSPL as primary task metrics; TL characterizes path efficiency, NE measures endpoint accuracy, and runtime reflects computational cost.

All variants use the same baseline checkpoints and differ only in their inference-time control settings; none requires retraining. Unless stated otherwise, DART-VLN denotes *decay+anti-loop* under deterministic argmax inference. We report *update-only*, *decay-only*, and *full-mode* as auxiliary ablations and stress tests. The default configuration uses $\lambda = 0.12$, $\alpha = 0.15$, $w_{\min} = 0.35$, $w_{\max} = 1.0$, $\beta_{\text{back}} = 0.22$, $\beta_{\text{rev}} = 0.06$, and $k = 2$. We use these hyperparameters in all DART-VLN comparisons without retraining or checkpoint selection. Write-side variants use $g_0 = 0.15$, $\lambda_n = 0.35$, $\lambda_a = 0.20$, $\lambda_c = 0.15$, $g_{\min} = 0.05$, $g_{\max} = 0.85$, $T_{\max} = 20$, and $r_{\text{match}} = 0.75$. Runtime is reported only for GridMM and its variants, evaluated with the same implementation on an NVIDIA GeForce RTX 5070 Ti; cross-paper runtime is not directly comparable. The reduction mainly reflects lower memory-access and downstream inference pressure, as decay concentrates readout on a distilled set of informative slots.

B. Main Results on R2R and REVERIE

Tables I and II show a consistent pattern: *decay-only* preserves or improves task performance while reducing runtime, whereas *decay+anti-loop* achieves the best balance among the GridMM variants. We discuss the main trends below and return to the stress-test variants in Sec. IV-C.

R2R. Table I reports results on both *val unseen* and *test unseen*. On this benchmark, *decay-only* already shows the value of conservative read-side control: compared with GridMM, it improves NE from 2.83 to 2.59 on *val unseen* and from 3.35 to 3.19 on *test unseen*, while also reducing runtime from 937.99 s to 742.61 s and from 2312.52 s to 1621.34 s, respectively. Its SR and SPL are maintained or slightly improved, suggesting that read-side decay reduces the influence of stale memory while improving inference efficiency.

Among the GridMM variants, *decay+anti-loop* achieves the best overall balance on R2R. It produces the shortest

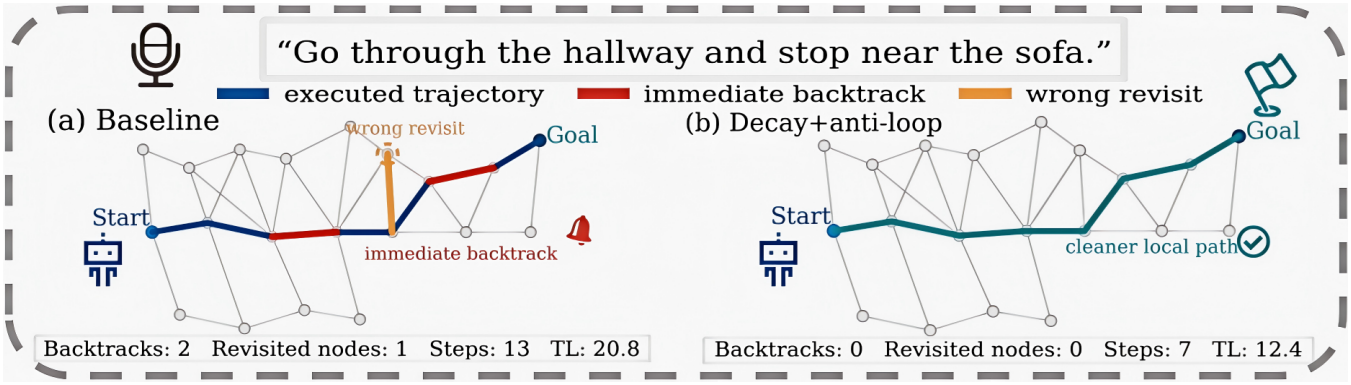


Fig. 2. Example in which *decay+anti-loop* avoids immediate backtracking and follows a shorter path.

TABLE II

RESULTS ON THE REVERIE VAL-UNSEEN SPLIT.

Methods	Val Unseen						Runtime(s)↓
	Navigation				Grounding		
	TL↓	OSR↑	SR↑	SPL↑	RGS↑	RGSPL↑	
VLNBERT [6]	16.75	35.08	30.63	24.90	18.72	15.23	–
AirBERT [7]	18.71	34.51	27.88	21.83	18.19	14.16	–
HOP [21]	16.46	36.24	31.78	26.15	18.82	15.73	–
HAMT [8]	**14.08**	36.84	32.91	30.23	18.90	17.26	–
TD-STP [22]	–	39.48	34.89	27.34	21.13	16.52	–
DUET [4]	22.12	51.08	46.96	33.77	32.12	23.01	–
BEVBert [5]	–	56.38	51.79	36.35	34.70	24.46	–
GridMM [3]	23.20	57.48	51.37	36.47	34.57	24.56	4329.67
update-only	23.77	57.12	50.84	35.85	33.98	23.80	4276.31
decay-only	23.15	**58.12**	51.98	36.60	34.68	24.72	2998.49
full-mode	23.18	57.94	51.80	36.45	34.65	24.68	4603.40
decay+anti-loop	**21.57**	57.99	**52.34**	**37.53**	**35.37**	**25.44**	**1497.98**

Note: Runtime is reported only for GridMM and its variants under the same setting; the official test unseen server is unavailable.

TABLE III

BEHAVIORAL ANALYSIS ON THE R2R AND REVERIE VAL-UNSEEN SPLITS.

Methods	R2R Val Unseen		
	Backtrack Rate(%)↓	Avg. Steps↓	Avg. TL↓
GridMM (baseline)	2.30	6.02	13.27
decay-only	3.51	6.00	13.29
decay+anti-loop	**2.01**↓	**5.90**↓	**12.41**↓
Methods	REVERIE Val Unseen		
	Backtrack Rate(%)↓	Avg. Steps↓	Avg. TL↓
GridMM (baseline)	8.43	8.52	23.20
decay-only	8.45	8.52	23.15
decay+anti-loop	**5.99**↓	**8.46**↓	**21.57**↓

Note: Backtrack rate is the percentage of executed actions that return to the viewpoint from two steps earlier; Avg. Steps counts actions before STOP. Because the default configuration uses only a weak revisit penalty, the observed changes are dominated by immediate-backtracking suppression.

trajectories and lowest runtime on both splits while improving SR/SPL from 73/62 to 74/63 on *test unseen*. Its test NE is slightly worse than GridMM (3.38 versus 3.35), so the result reflects a better balance between task performance and efficiency rather than a uniform improvement in endpoint accuracy.

REVERIE. Table II shows the same qualitative trend on *val unseen*. *decay-only* slightly improves navigation and grounding metrics while reducing runtime, indicating that the benefits of read-side memory control extend to grounding-related metrics. Among the GridMM variants, *decay+anti-loop* achieves the strongest overall performance, improving SR/SPL/RGSPL from 51.37/36.47/24.56 to 52.34/37.53/25.44 and reducing runtime from 4329.67 s to 1497.98 s. Together, the two benchmarks show that memory decay provides consistent improvements and that anti-loop regularization further reduces trajectory length and runtime.

C. Ablations and Stress-Test Variants

The comparison among *update-only*, *decay-only*, *full-mode*, and *decay+anti-loop* clarifies the role of each component. Among the single-module variants, *decay-only* consistently preserves or improves task metrics while reducing runtime. By contrast, *update-only* and *full-mode* rewrite memory before readout and show less consistent gains, suggesting that aggressive write-side intervention can disturb representations already used by the frozen policy.

decay+anti-loop builds on this read-side improvement with a narrow action-side controller that reduces local reversals, trajectory length, and runtime. The ablation supports the main design principle: with a frozen backbone, well-scoped test-time control is more reliable than heavier intervention.

D. Navigation Behavior and Efficiency Analysis

Anti-loop regularization is not meant to guarantee monotonic gains on all endpoint metrics; its direct value is local trajectory control. Figure 2 illustrates this effect, while Table III quantifies it through backtracking rate, average action count, and trajectory length.

On R2R, *decay-only* leaves local reversal behavior largely unchanged: its average action count changes only marginally, and its immediate-backtracking rate remains higher than that of the baseline. This is consistent with its role as a read-side denoising rule rather than a motion regularizer. By contrast, adding anti-loop regularization reduces the immediate-backtracking rate from 3.51% to 2.01% while also decreasing the average number of actions and trajectory length.

REVERIE exhibits the same pattern. The baseline and *decay-only* remain nearly identical in local reversal behavior, whereas *decay+anti-loop* reduces the backtracking rate from 8.45% to 5.99% and follows shorter trajectories. This consistency across benchmarks supports the intended role of

anti-loop regularization as a graph-local controller rather than a general planning mechanism.

V. Conclusion

We have presented DART-VLN, a training-free plug-in control layer for discrete VLN pipelines with explicit memory. When instantiated on a GridMM-based navigator, DART-VLN improved navigation performance and efficiency on R2R and REVERIE without retraining or architectural changes. Read-side decay preserved or improved task performance while reducing runtime, whereas anti-loop regularization further reduced local backtracking and trajectory length. Together, these findings highlight the practical value of conservative test-time control for improving inference behavior and efficiency in frozen memory-based navigators.

Beyond the aggregate results, our behavioral analysis clarified how the two modules contributed at different stages of inference: decay stabilized memory readout, whereas anti-loop regularization improved local trajectory control. Their combination offers a compact and interpretable intervention for frozen navigators.

Our experiments focused on a GridMM-based instantiation, which provided a controlled setting for isolating these effects. Future work will examine transfer to other explicit-memory backbones and hyperparameter settings, as well as stochastic decoding and continuous navigation, to assess the broader applicability of conservative test-time control.

References

[1] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. van den Hengel, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3674–3683.

[2] Y. Zhang, Z. Ma, J. Li, Y. Qiao, Z. Wang, J. Chai, Q. Wu, M. Bansal, and P. Kordjamshidi, "Vision-and-language navigation today and tomorrow: A survey in the era of foundation models," *Trans. Mach. Learn. Res.*, 2024.

[3] Z. Wang, X. Li, J. Yang, Y. Liu, and S. Jiang, "GridMM: Grid memory map for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 15 579–15 590.

[4] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16 537–16 547.

[5] D. An, Y. Qi, Y. Li, Y. Huang, L. Wang, T. Tan, and J. Shao, "BEVBert: Topo-metric map pre-training for language-guided navigation," *arXiv:2212.04385*, 2022.

[6] Y. Hong, Q. Wu, Y. Qi, C. Rodriguez-Opazo, and S. Gould, "VLN-BERT: A recurrent vision-and-language BERT for navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1643–1653.

[7] P.-L. Guhur, M. Tapaswi, H. Chen, I. Laptev, and C. Schmid, "Airbert: In-domain pretraining for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1614–1623.

[8] S. Chen, P.-L. Guhur, C. Schmid, and I. Laptev, "History aware multimodal transformer for vision-and-language navigation," in *Adv. Neural Inf. Process. Syst.*, 2021.

[9] C. Gao, X. Peng, M. Yan, H. Wang, L. Yang, H. Ren, H. Li, and S. Liu, "Adaptive zone-aware hierarchical planner for vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14 911–14 920.

[10] X. Dong, H. Zhao, J. Gao, H. Li, X. Ma, Y. Zhou, F. Chen, and J. Liu, "SE-VLN: A self-evolving vision-language navigation framework based on multimodal large language models," *arXiv:2507.13152*, 2025.

[11] J. Gao, R. Liu, and W. Wang, "3D Gaussian map with open-set semantic grouping for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 9252–9262.

[12] S. Zhang, Y. Qiao, Q. Wang, Z. Yan, Q. Wu, Z. Wei, and J. Liu, "COSMO: Combination of selective memorization for low-cost vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 5511–5522.

[13] J. Krantz, S. Banerjee, W. Zhu, J. Corso, P. Anderson, S. Lee, and J. Thomason, "Iterative vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14 921–14 930.

[14] C.-Y. Ma, Z. Wu, G. AlRegib, C. Xiong, and Z. Kira, "The regretful agent: Heuristic-aided navigation through progress estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6732–6740.

[15] H. Wang, W. Liang, L. Van Gool, and W. Wang, "DREAMWALKER: Mental planning for continuous vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 10 839–10 849.

[16] P. Xu, X. Gong, and Y. Mu, "NavQ: Learning a Q-model for foresighted vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 6327–6341.

[17] G. Georgakis, K. Schmeckpeper, K. Wanchoo, S. Dan, E. Miltsakaki, D. Roth, and K. Daniilidis, "Cross-modal map learning for vision and language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15 460–15 470.

[18] N. Rajabi and J. Kosecka, "TRAVEL: Training-free retrieval and alignment for vision-and-language navigation," *arXiv:2502.07306*, 2025.

[19] H. Ko, S. Kim, G. Oh, J. Yoon, H. Lee, S. Jang, S. Kim, and S. Kim, "Active test-time vision-language navigation," in *Adv. Neural Inf. Process. Syst.*, 2025.

[20] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 104–120.

[21] Y. Qiao, Y. Qi, Y. Hong, Z. Yu, P. Wang, and Q. Wu, "HOP+: History-enhanced and order-aware pre-training for vision-and-language navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 7, pp. 8524–8537, 2023.

[22] Y. Zhao, J. Chen, C. Gao, W. Wang, L. Yang, H. Ren, H. Xia, and S. Liu, "Target-driven structured transformer planner for vision-language navigation," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 4194–4203.

[23] Y. Qi, Q. Wu, P. Anderson, X. Wang, W. Y. Wang, C. Shen, and A. van den Hengel, "REVERIE: Remote embodied visual referring expression in real indoor environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9979–9988.